

Report conversation

hey, I need help slightly changing a ruby plugin I wrote.

the following is the current code for a jekyll plugin that allows me to fetch posts from an external rss feed when `rss_url` is specified.

```
ruby
require 'feedjira'
require 'httparty'
require 'jekyll'

module ExternalPosts
  class ExternalPostsGenerator < Jekyll::Generator
    safe true
    priority :high

    def generate(site)
      if site.config['external_sources'] != nil
        site.config['external_sources'].each do |source|
          p "Fetching external posts from #{source}"
          xml = HTTParty.get(src[source['rss_url']])
          feed = Feedjira.parse(xml)
          feed.entries.each do |e|
            p "...fetching #{e.url}"
            slug = e.title.downcase.strip.gsub(' ', '-').gsub('.', '-')
            path = site.in_source_dir("_posts/#{slug}")
            doc = Jekyll::Document.new(
              path, { :site => site, :collection => 'posts' }
            )
            doc.data['external_source'] = source['name']
            doc.data['feed_content'] = e.content
            doc.data['title'] = "#{e.title}"
            doc.data['description'] = e.summary
            doc.data['date'] = e.published
            doc.data['redirect'] = e.url;
            site.collections['posts'].docs << doc
          end
        end
      end
    end
  end
end
```

```
        end
      end
    end
  end

end
```

here's an example `site.config` that the code above uses:

```
yaml
external_sources:
  - name: medium.com
    rss_feed: <url-to-medium-feed>
```

now, I would like to update the plugin such that it can fetch also some posts from manually specified urls instead of an RSS feed. here's an example of a config that I would like to support:

```
yaml
external_sources:
  - name: medium.com
    urls:
      - <url-to-post1>
      - <url-to-post2>
      - <url-to-post3>
```

can you please help me update the plugin?

Certainly! To modify your Jekyll plugin to support fetching posts from manually specified URLs in addition to RSS feeds, you'll need to add a section to your code that handles these URLs. Here's an updated version of your plugin with the necessary changes:

1. Check for the presence of the `urls` key in each source.
2. For each URL, fetch the content. You might need a way to extract the relevant post information from these URLs, as they won't be in a standardized format like RSS. This

could be a challenging part, depending on the structure of the web pages you're fetching. I'll add a placeholder method `fetch_content_from_url` for this purpose.

Please note that without knowing the exact format of the web pages you're fetching, it's hard to write a precise method for extracting the post content. You'll need to tailor this part of the code to fit your specific needs.

Here's the updated plugin code:

ruby

 Copy code

```
require 'feedjira'
require 'httparty'
require 'jekyll'
require 'nokogiri' # Add if needed for HTML parsing

module ExternalPosts
  class ExternalPostsGenerator < Jekyll::Generator
    safe true
    priority :high

    def generate(site)
      if site.config['external_sources'] != nil
        site.config['external_sources'].each do |src|
          if src['rss_url']
            fetch_from_rss(site, src)
          elsif src['urls']
            fetch_from_urls(site, src)
          end
        end
      end
    end

    def fetch_from_rss(site, src)
      p "Fetching external posts from #{src['name']}:"
      xml = HTTParty.get(src['rss_url']).body
      feed = Feedjira.parse(xml)
      process_entries(site, src, feed.entries)
    end

    def fetch_from_urls(site, src)
```

```

src['urls'].each do |url|
  p "...fetching #{url}"
  content = fetch_content_from_url(url)
  # Here you need to extract title, summary, content, etc. from 'content'
  # The following is a placeholder for the real extraction logic.
  # e.g., title = content[:title], summary = content[:summary]
  create_document(site, src['name'], url, content)
end
end

def fetch_content_from_url(url)
  # Fetch and parse the HTML content from the URL
  # Return a hash or similar structure containing the post's details
  # This is a placeholder method
  html = HTTParty.get(url).body
  # Use Nokogiri or similar library to parse HTML and extract post details
  # For example: Nokogiri::HTML(html).at('h1').text for the title
  # return { title: ..., content: ..., summary: ..., published: ... }
end

def process_entries(site, src, entries)
  entries.each do |e|
    p "...fetching #{e.url}"
    create_document(site, src['name'], e.url, {
      title: e.title,
      content: e.content,
      summary: e.summary,
      published: e.published
    })
  end
end

def create_document(site, source_name, url, content)
  slug = content[:title].downcase.strip.gsub(' ', '-').gsub(/[^\w-]/, '')
  path = site.in_source_dir("_posts/#{slug}.md")
  doc = Jekyll::Document.new(
    path, { :site => site, :collection => site.collections['posts'] }
  )
  doc.data['external_source'] = source_name
  doc.data['feed_content'] = content[:content]
end

```

```
doc.data['title'] = content[:title]
doc.data['description'] = content[:summary]
doc.data['date'] = content[:published]
doc.data['redirect'] = url
site.collections['posts'].docs << doc
end
end
end
```

Remember to adapt the `fetch_content_from_url` method to suit the structure of the web pages you're fetching. This method should return a hash containing the title, content, summary, and published date of each post, extracted from the HTML of the web page.

perfect, that's a good starting point. I still need help implementing `fetch_from_urls` and `fetch_content_from_url` functions as I'm not very familiar with ruby. I can tell you that we can probably grab `title` and `description` from the webpage metadata, defined between the `<head></head>` tags. and for the content, maybe we can grab everything inside the `<body></body>` tags? could you please implement these two functions for me?

Absolutely, I can help with that. To extract the `title`, `description`, and content from the web pages, we'll use the Nokogiri gem, a Ruby library for parsing HTML and XML. Make sure you have Nokogiri installed in your environment. If not, you can add it to your Gemfile and run `bundle install`.

Here's how you can implement the `fetch_from_urls` and `fetch_content_from_url` methods:

1. `fetch_from_urls` will iterate over the URLs and call `fetch_content_from_url` for each one.

2. `fetch_content_from_url` will fetch the webpage, parse it, and extract the `title` and `description` from the metadata in the `<head>` section and the entire content within the `<body>` tags.

Here's the implementation:

ruby

 Copy code

```
require 'nokogiri'

# ...

def fetch_from_urls(site, src)
  src['urls'].each do |url|
    p "...fetching #{url}"
    content = fetch_content_from_url(url)
    create_document(site, src['name'], url, content)
  end
end

def fetch_content_from_url(url)
  html = HTTParty.get(url).body
  parsed_html = Nokogiri::HTML(html)

  title = parsed_html.at('head title')&.text || ''
  description = parsed_html.at('head meta[name="description"]')&.attr('content') || ''
  body_content = parsed_html.at('body')&.inner_html || ''

  {
    title: title,
    content: body_content,
    summary: description,
    published: Time.now # Placeholder for the published date
  }
end

# ...
```

A few things to note about this implementation:

1. The `title` is fetched from the `<title>` tag within the `<head>` section.

2. The `description` is fetched from a `<meta>` tag with the attribute `name="description"` within the `<head>` section.
3. The content is taken from the entire inner HTML of the `<body>` tag.
4. The `published` date is set to the current time as a placeholder, since this information is not available in the metadata. You might want to adjust this based on your requirements.

Remember, the effectiveness of this method depends on the consistency of the web pages you are fetching. If the structure of these pages varies significantly, you might need a more robust or tailored solution for content extraction.

nice, thanks! looks like you put a placeholder for the published date as the current date. do you know if it is possible to extract information about when the webpage was published somehow?

Extracting the publication date of a webpage is a bit more challenging because there isn't a standard method for websites to display this information. However, there are a few approaches you can try:

1. **Metadata Tags:** Some websites use metadata tags within the `<head>` section of the HTML to specify the publication date. Common tag names include `article:published_time`, `og:published_time`, or simply `published`.
2. **Article Tags:** If the webpage follows semantic HTML5 standards, the publication date might be included in an `<article>` tag, possibly within a `<time>` element.
3. **URL or Page Content:** Occasionally, the publication date is embedded in the URL or visibly displayed on the page. This would require custom parsing logic for each website format.
4. **HTTP Headers:** Sometimes, the HTTP headers of the webpage contain a `Last-Modified` field. This isn't necessarily the publication date but can sometimes serve as a rough estimate.

For a more robust implementation, you might need to combine several of these methods and include fallbacks. Keep in mind that the accuracy of these methods can vary greatly

from site to site. Here's an updated version of the `fetch_content_from_url` function implementing some of these strategies:

ruby

 Copy code

```
def fetch_content_from_url(url)
  response = HTTParty.get(url)
  html = response.body
  parsed_html = Nokogiri::HTML(html)

  title = parsed_html.at('head title')&.text || ''
  description = parsed_html.at('head meta[name="description"]')&.attr('content') || ''
  body_content = parsed_html.at('body')&.inner_html || ''
  published = parse_published_date(parsed_html, response.headers)

  {
    title: title,
    content: body_content,
    summary: description,
    published: published
  }
end

def parse_published_date(parsed_html, headers)
  # Try to extract from metadata
  published_meta = parsed_html.at('head meta[property="article:published_time"]')
  return published_meta if published_meta

  # Try to extract from article tag
  published_article = parsed_html.at('article time')&.attr('datetime')
  return published_article if published_article

  # Fallback to Last-Modified header
  headers['Last-Modified'] || Time.now
end
```

This code attempts to extract the publication date from various sources and falls back to the current time if none are found. Keep in mind that the effectiveness of this method

will depend on the specific websites you're scraping, and you might need to tailor the logic to suit your needs.

ok, I have a simpler solution: let's require the user who provides the list of urls to also specify `published_date` field in the yaml config for each entry and use that as published date. the new config would look something like this:

yaml

```
external_sources:
  - name: medium.com
    posts:
      - url: <url-to-post1>
        published_date: <published-date1>
      - url: <url-to-post2>
        published_date: <published-date2>
      - url: <url-to-post3>
        published_date: <published-date3>
```

could you update the code please?

Certainly, modifying the plugin to use a user-provided `published_date` for each URL in the YAML configuration is a practical approach. This change will make the implementation more straightforward and reliable. I'll update the code accordingly.

Here's the revised version of your plugin with the new configuration structure:

ruby

 Copy code

```
require 'nokogiri'
```

```
# ...
```

```
def fetch_from_urls(site, src)
  src['posts'].each do |post|
```

```

    p "...fetching #{post['url']}"
    content = fetch_content_from_url(post['url'])
    content[:published] = post['published_date'] # Use the provided published date
    create_document(site, src['name'], post['url'], content)
  end
end

def fetch_content_from_url(url)
  html = HTTParty.get(url).body
  parsed_html = Nokogiri::HTML(html)

  title = parsed_html.at('head title')&.text || ''
  description = parsed_html.at('head meta[name="description"]')&.attr('content') || ''
  body_content = parsed_html.at('body')&.inner_html || ''

  {
    title: title,
    content: body_content,
    summary: description
    # Note: The published date is now added in the fetch_from_urls method
  }
end

# ...

```

In this updated version, the `fetch_from_urls` method now also reads the `published_date` from the YAML configuration and adds it to the content hash. The `fetch_content_from_url` method remains the same but no longer handles the published date, as this is now provided in the configuration.

This approach requires that the users of your plugin specify the `published_date` for each post in their YAML configuration, ensuring that the date is accurate and tailored to each specific post.

ok, this is a minor detail, but I don't like that ruby prints log messages in quotation marks when we call `p`

"...fetching #{post['url']}". how do I make it print normal log messages without quotations?

In Ruby, the `p` method is used for debugging and it prints the value in a way that's helpful for debugging, including the quotation marks around strings. To print log messages without quotation marks, you can use `puts` instead of `p`. The `puts` method is more suitable for user-facing output as it prints the string representation of an object to the console without additional formatting like quotation marks.

Here's how you can change your code to use `puts` for logging:

ruby

 Copy code

```
def fetch_from_urls(site, src)
  src['posts'].each do |post|
    puts "...fetching #{post['url']}" # Changed 'p' to 'puts' here
    content = fetch_content_from_url(post['url'])
    content[:published] = post['published_date'] # Use the provided published date
    create_document(site, src['name'], post['url'], content)
  end
end

# Rest of your code remains the same...
```

This will print the log messages without the quotation marks, providing cleaner output for the user.

cool, thanks. I'm testing it now, and running into the following error:

```
.rbenv/versions/3.2.2/lib/ruby/gems/3.2.0/gems
```

is this because I might have incorrectly specified the date in the yaml file? I wrote `published_date: 2017-10-11`, which seems like a reasonable date format... can you help?

The error you're encountering is likely due to how Ruby is interpreting the date format from your YAML file. In Ruby, YAML-parsed dates are typically converted to `Date` objects, not strings or integers. Jekyll might be expecting a different format (like a timestamp or a string) for the `published_date` field.

To resolve this issue, you can convert the date to a string in the YAML file, and then parse it back into a `Date` object in your Ruby code. This way, you can ensure that Jekyll receives the date in a format it can work with.

Here's how you can modify your code:

1. In your YAML file, enclose the date in quotes to ensure it's treated as a string:

yaml

 Copy code

```
external_sources:
  - name: medium.com
    posts:
      - url: <url-to-post1>
        published_date: "2017-10-11"
      - url: <url-to-post2>
        published_date: "2018-05-22"
  # ... other posts
```

2. In your Ruby code, parse the string into a `Date` object:

ruby

 Copy code

```
require 'date'

# ...

def fetch_from_urls(site, src)
  src['posts'].each do |post|
    puts "...fetching #{post['url']}"
    content = fetch_content_from_url(post['url'])
    content[:published] = Date.parse(post['published_date'])
    create_document(site, src['name'], post['url'], content)
  end
end
```

```
# ...
```

By quoting the dates in the YAML file, they are explicitly treated as strings. The `Date.parse` method in Ruby then converts these strings into `Date` objects, which should be compatible with Jekyll's expectations. This approach should help you avoid the `NoMethodError` you're encountering.

```
hmm, looks like that didn't help. still getting this error:
.rbenv/versions/3.2.2/lib/ruby/gems/3.2.0/gem
```

```
thing.respond_to?(:date) && !@site.full
```

```
Did you mean? to_s
```

```
what's the problem?
```

The error message you're encountering suggests that Jekyll is expecting a date in a format that responds to the `to_i` method, which typically converts a time-based object to an integer representing the number of seconds since the Unix epoch. The `Date` object in Ruby doesn't directly support the `to_i` method, which is why you're seeing this error.

In Jekyll, dates are usually handled as `Time` objects, not `Date` objects. The `Time` object in Ruby supports the `to_i` method, which is likely what Jekyll is expecting. To resolve this issue, you should convert the parsed dates to `Time` objects.

Here's how to update your code:

```
ruby
```

 Copy code

```
require 'time'
```

```
# ...
```

```
def fetch_from_urls(site, src)
  src['posts'].each do |post|
```

```

puts "...fetching #{post['url']}"
content = fetch_content_from_url(post['url'])
content[:published] = Time.parse(post['published_date']).utc
create_document(site, src['name'], post['url'], content)
end
end

# ...

```

In this updated version, `Time.parse` is used instead of `Date.parse`, and I've added `.utc` to convert the time to Coordinated Universal Time (UTC). This should provide a `Time` object that is compatible with Jekyll's requirements, resolving the `NoMethodError`.

amazing, it works now, you nailed it! one last thing: I don't like that I have to specify date in the yaml file in quotation marks like "2017-10-11". if I don't add quotation marks, I get the following error:

```
.rbenv/versions/3.2.2/lib/ruby/3.2.0/time.rb
```

```
d = Date._parse(date, comp)
```

could you update the code to allow specifying dates in the yaml without quotations?

Certainly! When dates are specified in a YAML file without quotation marks, they are typically parsed as `Date` objects in Ruby, rather than as strings. To handle this scenario, you can modify your code to check the type of the `published_date` and process it accordingly.

Here's how you can update the `fetch_from_urls` method to handle both `String` and `Date` types for `published_date`:

```
ruby
```

 Copy code

```

def fetch_from_urls(site, src)
  src['posts'].each do |post|
    puts "...fetching #{post['url']}\""
    content = fetch_content_from_url(post['url'])
    published_date = post['published_date']

    # Convert to Time object depending on the type of 'published_date'
    content[:published] = case published_date
                          when String
                            Time.parse(published_date).utc
                          when Date
                            published_date.to_time.utc
                          else
                            raise "Invalid date format for #{published_date}"
                          end

    create_document(site, src['name'], post['url'], content)
  end
end

```

In this updated version, the code first checks the type of `published_date`. If it's a `String`, it uses `Time.parse` to convert it to a `Time` object. If it's a `Date` object (which happens if you don't use quotation marks in the YAML), it calls `to_time` to convert it to a `Time` object. In both cases, `.utc` is called to ensure the time is in UTC.

This modification should allow you to specify dates in the YAML file with or without quotation marks.

thanks. the code looks a bit ugly. let's move the casing logic into a separate function?

Absolutely, moving the date parsing logic into a separate function will make your code cleaner and more maintainable. I'll create a new method called `parse_published_date`

which handles the conversion of the `published_date` to a `Time` object. This method will take the `published_date` as an argument and return the appropriate `Time` object.

Here's the refactored code:

ruby

 Copy code

```
def fetch_from_urls(site, src)
  src['posts'].each do |post|
    puts "...fetching #{post['url']}"
    content = fetch_content_from_url(post['url'])
    content[:published] = parse_published_date(post['published_date'])
    create_document(site, src['name'], post['url'], content)
  end
end

def parse_published_date(published_date)
  case published_date
  when String
    Time.parse(published_date).utc
  when Date
    published_date.to_time.utc
  else
    raise "Invalid date format for #{published_date}"
  end
end

# Rest of your code...
```

In this structure, the `parse_published_date` method abstracts away the details of converting the `published_date` to a `Time` object, making the `fetch_from_urls` method more readable and focusing it on its primary responsibility. The `parse_published_date` method handles different types of date formats and raises an